IBM

IBM Aspera®

Aspera Orchestrator Portlet Development Guide

IBM Aspera

2026/11/03



BUILD • AUTOMATE • ORCHESTRATE

Contents

1 Introduction **2**
1.1 Terminology . . . 2

2 Quick start **4**
2.1 Source . . . 4
2.2 Install and run . . . 5

3 Portlet folder **6**
3.1 Names . . . 6
3.2 Files . . . 6
3.3 Metadata file . . . 7

4 Portlet class **8**
4.1 Structure . . . 8
4.2 Methods . . . 8
4.3 Types . . . 8
4.4 Runtime values . . . 9
4.5 Parameter `refresh` . . . 9

5 Views and actions **10**
5.1 Display in a frame . . . 10
5.2 Main view . . . 10
5.3 Configuration partial . . . 11
5.4 JavaScript . . . 11
5.5 Actions . . . 12

6 Installation and updates **13**
6.1 Installing a portlet . . . 13
6.2 Importing and exporting . . . 13
6.3 Updating a portlet . . . 13
6.4 Upgrading a legacy portlet . . . 14

7 Testing and debugging **15**
7.1 Known issues . . . 15

8 Security **17**
8.1 Permissions . . . 17
8.2 Security of actions . . . 17

9 Checklist **19**

10 Annex **20**
10.1 Comparison with plugins . . . 20
10.2 Custom portlets . . . 20
10.3 Patterns in the shipped portlets . . . 21
10.4 Learning Ruby and Rails . . . 21

Chapter 1

Introduction

This guide explains how to write a portlet for IBM Aspera Orchestrator: a Ruby class, a view, a configuration partial and a metadata file, and how to install, test and update it. A portlet is a widget of the Orchestrator dashboard.

It is written for people and for AI assistants. Each rule gives the exact name, method or value expected by Orchestrator.

The content is derived from the Orchestrator 4.1.6 source code (build 2036, Ruby 3.4, Rails 8.1).

How it was verified:

- On an Orchestrator 4.1.6.2036 server: the example portlet of Quick start was imported with Import Portlet, chosen and configured in a dashboard frame, displayed, and its action called with GET and POST. In Chrome, the Ping link (`ajaxUpdate`) and the automatic refresh worked with no JavaScript error. The behaviors of import (file name, collision), enable, disable, restart, the version API and the CSRF token described below were reproduced on this server.
- From the source code only: the security notes, the custom portlets, the legacy upgrade script, and the counts of the Annex.

Related documents:

- Plugin Development Guide: the same for action plugins. Portlets and plugins share several mechanisms, listed in Comparison with plugins.
- Workflow Authoring Guide: how to write workflows, whose work orders a portlet can display.

The source of the shipped portlets is the best set of examples: folder `/opt/aspera/orchestrator/portlets` of an Orchestrator server, one folder per portlet. `orchestrator_information` is the simplest, `user_tasks` has parameters and a permission check, `csv_editor` has uploads, a modal dialog and several actions, `queue_management` has pagination and `helper_method`.

The repository `aspera-orchestrator-secret-reference` contains the portlet `secret_dashboard` as a worked example: it uses a plugin (`SecretCore`), has four sections with AJAX actions, an admin-only check in every action, and a full RSpec test suite that runs outside Rails with stubs of `Portlet`, `PortletsController`, `Workflow`, `WorkStep` and `User`.

1.1 Terminology

Term	Meaning
Portlet	A Ruby class that inherits from `Portlet`, for example `HelloBanner`, and its folder. Displays one widget.
Portlet ID	The folder name in snake_case, for example `hello_banner`. The class name is `portlet_id.camelize`.
Frame	A zone of a dashboard page. A frame shows one portlet, or is split horizontally or vertically into sub-frames. Stored in table `frames`.
Dashboard	The frames of a user or of a group, shown by menu Dashboard.
Parameter	A named, typed setting of a portlet, edited in the configuration of the frame and stored in the frame, not in the portlet.
Runtime value	The value of a parameter during one display: the frame value, or the default.

Term	Meaning
Portlet action	A public method of the portlet controller, called from the view with an HTTP request, for example to load a table page.
Custom portlet	A portlet defined in the web interface (Ruby and ERB text in the database) instead of files. See Custom portlets.
Stock folder	`/opt/aspera/orchestrator/portlets/<id>`: shipped portlets.
User folder	`<config_dir>/portlets/<id>`, by default `/opt/aspera/var/config/orchestrator/portlets/<id>`: portlets written or imported by the user.

Chapter 2

Quick start

The following portlet `hello_banner` shows a greeting with the time, and the most recent work orders. It has four parameters (`greeting`, `uppercase`, `max_orders`, `refresh`), a configuration partial, and one action (`ping`) called with `ajaxUpdate`.

2.1 Source

```
hello_banner/
├── hello_banner.rb        class HelloBanner and class HelloBannerController
├── hello_banner.erb       main view
├── _hello_banner.erb      configuration partial
└── metadata.yml           version history
```

`hello_banner.rb`:

```
# Shows a message, optionally in upper case, with a counter of work orders.
class HelloBanner < Portlet
  after_initialize :set_portlet_name_and_desc
  before_save      :set_portlet_name_and_desc
  def set_portlet_name_and_desc
    self.name = "Hello banner"
    self.description = "Example portlet of the portlet development guide."
  end

  def parameters_spec
    return {
      "refresh" => TYPE_INT,
      "greeting" => TYPE_STRING,
      "uppercase" => TYPE_FLAG,
      "max_orders" => TYPE_INT
    }
  end

  def defaulted_parameters
    return { "refresh" => 60, "greeting" => "Hello", "uppercase" => false, "max_orders" => 5 }
  end

  def message
    text = "#{greeting}, #{Time.now.utc.strftime('%H:%M:%S')} UTC"
    return uppercase ? text.upcase : text
  end

  def recent_orders
    return WorkOrder.order(initiate_date: :desc).limit(max_orders).to_a
  end
end

class HelloBannerController < PortletsController
```

```
  def ping
    render plain: "pong #{@portlet.greeting} from frame #{@frame_id}"
  end
end
```

`hello_banner.erb`:

```
<main class="container-fluid" aria-labelledby="hello_banner-heading">
  <h4 class="page_title" id="hello_banner-heading"><%= h @portlet.message %></h4>
  <table class="table table-condensed">
    <% @portlet.recent_orders.each do |order| %>
      <tr><td><%= h order.name %></td><td><%= h order.status %></td></tr>
    <% end %>
  </table>
  <a href="#" onclick="ajaxUpdate('<%= raw portlet_action_path(portlet: "hello_banner",
  ↪  portlet_action: "ping", frame_id: @frame_id, frame_html_id: @frame_html_id) %>', 'GET', {
  ↪  target_id: '#<%= @frame_html_id %> #hello_ping' }); return false">Ping</a>
  <span id="hello_ping"></span>
</main>
```

`_hello_banner.erb`:

```
<li>
  <label>Greeting</label>
  <input type="text" <%= portlet_input_name_and_value(@portlet, :greeting) %> <%= style_long %>/>
</li>
<%= draw_portlet_flag_list_element("uppercase", "Upper case") %>
<li>
  <label>Orders shown</label>
  <input type="text" <%= portlet_input_name_and_value(@portlet, :max_orders) %> <%= style_short %>/>
</li>
<li>
  <label>Refresh rate</label>
  <input type="text" <%= portlet_input_name_and_value(@portlet, :refresh) %> <%= style_short %>/>
</li>
```

`metadata.yml`:

```
---
:revision_history:
  - :author: "<AUTHOR>"
    :change_description: Initial release
    :version: 1.0.0
    :date: "2026-10-30"
    :minimum_server_version: 4.1.0
```

2.2 Install and run

1. Pack the folder as `hello_banner.portlet`: `tar -czf hello_banner.portlet hello_banner`. The file name must be exactly `<portlet id>.portlet` (see Importing and exporting).
2. In the web interface, open Engine > Portlets, click Import Portlet, and select the file. Alternatively, copy the folder to `/opt/aspera/var/config/orchestrator/portlets/hello_banner` and use the slider of the portlet.
3. Enable the portlet with its slider, if the import did not already do so.
4. Open Dashboard, select a frame, choose Hello banner in Choose Portlet, set the parameters, and click Save Configuration.

A code change in an existing portlet needs a restart of the web server, `systemctl restart orchestrator_server`, see Updating a portlet.

Chapter 3

Portlet folder

3.1 Names

Item	Rule	Example
Folder	Portlet ID, snake_case.	`hello_banner`
Main file	`<id>.rb` in the folder. This exact name is loaded.	`hello_banner.rb`
Class	`id.camelize`, a subclass of `Portlet`.	`HelloBanner`
Controller class	`<Class>Controller`, a subclass of `PortletsController`, optional.	`HelloBannerController`
Main view	`<id>.erb`	`hello_banner.erb`
Configuration partial	`_<id>.erb`	`_hello_banner.erb`

Rules:

- The class is looked up with `Object.const_get(id.camelize)`. Orchestrator does not use `tableize` or `classify` for portlets, but the inflections of `config/initializers/inflections.rb` (for example `SSH`, `CGI`) still apply to `camelize`. Avoid an acronym in the ID.
- If the class is not defined after loading `<id>.rb`, enabling fails with ”definition of the portlet is missing”.
- `portlet_id` of a new object is `self.class.name` without a trailing `Portlet`, in snake_case, and `Portlet.generate` sets it to the folder name anyway. Do not override it: the class name must be `id.camelize` in all cases.
- A portlet ID must not collide with a class of Orchestrator. Do not name a portlet `User` or `Workflow`.
- The name (`self.name`) must be unique among portlets: Choose Portlet sends the name, and `Frame#select` finds the portlet with `Portlet.find_by(name:)`. The REST call `api/portlet_version` also takes the name.

3.2 Files

File	Required	Content
`<id>.rb`	Yes	The model class. The controller class can be in the same file.
`<id>.erb`	Yes	The view displayed in the frame. Rendered as `<id>/<id>` with layout `individual_portlet`.
`_<id>.erb`	Yes	The configuration partial: `<li>` elements shown in the configuration form of the frame. For a portlet with no setting, create an empty file: without it, choosing the portlet in a frame and Save Configuration fail with HTTP 500 (Missing partial).
`metadata.yml`	Recommended	The version history. Read by `Portlet.version` only.
`_*.erb`, `*.js`, `*.css`, images	No	Other partials, rendered as `<id>/<partial>`.

There is no migration, no icon and no help page: a portlet has no table of its own.

3.3 Metadata file

`metadata.yml` is a YAML file whose keys are symbols:

```yaml
---
:revision_history:
  - :author: "<AUTHOR>"
    :change_description: Initial release
    :version: 1.0.0
    :date: "2026-10-30"
    :minimum_server_version: 4.1.0
```

Key	Meaning
`:version`	`major.minor.revision`, optionally `.patch`.
`:change_description`, `:author`, `:date`	Free text.
`:minimum_server_version`, `:maximum_server_version`	Informational.

Limits of 4.1.6:

- `Portlet.version` reads the file, sorts the entries and returns the highest version as `[major, minor, revision, patch]`. It is used only by the REST call `GET api/portlet_version` (see Known issues).
- The server version is not checked for portlets, whereas it is for plugins. A portlet is enabled even if `:minimum_server_version` is higher than the server.
- A missing `metadata.yml` does not prevent enabling; only a call to `.version` raises `Errno::ENOENT`.

Chapter 4

Portlet class

4.1 Structure

```ruby
class MyPortlet < Portlet
  after_initialize :set_portlet_name_and_desc
  before_save      :set_portlet_name_and_desc
  def set_portlet_name_and_desc
    self.name = "My portlet"
    self.description = "What it shows."
  end

  def parameters_spec;      { "refresh" => TYPE_INT }; end
  def defaulted_parameters; { "refresh" => 60 };       end
end
```

`Portlet` is an Active Record model of table `portlets` (columns `name`, `description`, `active`, `install_source`, `portlet_id`, `parameters`, timestamps). All portlets share this table; there is no single-table inheritance.

4.2 Methods

Method	Required	Role
`set_portlet_name_and_desc`	Yes	Assigns `self.name` and `self.description`. Called by the two callbacks.
`parameters_spec`	No	Hash `name => TYPE_*`. Default: `{}`.
`defaulted_parameters`	No	Hash `name => default value`. Default: `{}`.
`view_template`	No	Template rendered by `PortletsController#view`. Default: `<id>/<id>`.
`url`, `generic_url`	No	Do not override. `Frame#select` reads `url` on the plain `Portlet` record returned by `Portlet.find_by(name:)`, not on your class, so an override has no effect.
`portlet_id`	No	Do not override. Default: class name without `Portlet` suffix, underscored.

Name and description must be literal strings: for a portlet that is not yet enabled, the page Engine > Portlets reads them with a regular expression on the lines `self.name = ...` and `self.description = ...`, removes the quotes and a trailing comment, and does not evaluate Ruby. A computed name is shown as raw text there.

4.3 Types

Constant	Value in a view	Conversion rule
`TYPE_STRING`	String	None.
`TYPE_INT`	Integer	`String#to_i`: `abc` gives `0`, `12abc` gives `12`.
`TYPE_FLAG`	`true`, `false` or `nil`	`true` only for the text `true` (any case). `1`, `on` and `yes` give `false`. A default of `false` gives `nil` (see below).
`TYPE_DATE`	`DateTime`	`DateTime.parse`; `nil` if the text cannot be parsed.
`TYPE_HASH`	Hash	Edited with `draw_variables_hash` / `script_for_variables_hash`.

The conversion is done by `MultiType.convert_value`. A parameter that is not in `parameters_spec` is still stored, as a raw string.

4.4 Runtime values

`Portlet.generate(id, params, frame_id)` creates the object and calls `set_runtime_values(params)`:

1. Start from `defaulted_parameters`.
2. A parameter in the request replaces its default only if it is not blank: a blank field keeps the default.
3. Convert each value with its type in `parameters_spec`.

A parameter with no default and no value is `nil`. `id`, `frame_id`, `frame_html_id` and every other URL parameter also become runtime values.

Reading a value: call the name as a method (`greeting`), or `runtime_value('greeting')`. `method_missing` looks, in order, at runtime values, instance variables, database attributes, then the keys of `parameters_spec` (giving `nil`). Anything else raises `NoMethodError`.

Gotchas:

- Database columns win. A parameter named `name`, `description`, `active` or `parameters` is hidden by the column: read it with `runtime_value('name')`, and prefer another name. Likewise `@portlet.id` is the row ID, not the URL parameter `id` (the portlet ID).
- A default value of `false` or `nil` gives the runtime value `nil`, because `set_runtime_value` keeps only values that are truthy. Test a flag with `if uppercase`, never with `== false`.
- `frame_id` is an `attr_accessor`, and `Portlet.generate` does not set it in `view` and `portlet_action`: use `@frame_id` in views and actions.
- `input_field(name)` returns `"__<name>"`, the name of the form field for a parameter.

4.5 Parameter `refresh`

All 23 shipped portlets have a parameter `refresh` (seconds) in `defaulted_parameters`; 19 also declare it in `parameters_spec` as `TYPE_INT`. Declare it in both. With a value above `0` and no sub-frame, the dashboard reloads the frame periodically while the page is visible. `-1` or `0` disables it. The layout `individual_portlet` shows ”Last refreshed at” when `refresh` is set and is `0` or more. If `refresh` is not in the link of the frame, the frame uses `defaulted_parameters['refresh']`.

Chapter 5

Views and actions

5.1 Display in a frame

How a portlet is shown:

1. A user opens a frame in Dashboard and selects a portlet in Choose Portlet, which sends the portlet name. `Frame#select` finds the row by name and stores `portlet_name`, `portlet_id`, `link = portlets/view?id=<id>` and `configuration_link = <portlet folder>/<id>`, for example `.../portlets/hello_banner/hello_banner`.
2. Save Configuration stores `link_parameters` (a Ruby hash as text, for example `{"greeting" => "Bonjour", "uppercase" => "true"}`) and appends `&<name>=<%= CGI::escape(<name>) %>` to the link for each saved parameter.
3. The page requests `FramesController#display`, which redirects to the expanded URL, with `frame_id` and `frame_html_id`, for example `portlets/view?id=hello_banner&greeting=Bonjour&uppercase=true&max_orders=3&refresh=30&frame_id=2&frame_html_id=dashboard_2`.
4. `PortletsController#view` merges `params` and `link_parameters`, calls `Portlet.generate`, and renders `view_template` in layout `individual_portlet`.
5. If `refresh` is above `0`, `FramesDisplay.loadSubframe` calls `startAjaxAutoUpdates` to reload the frame.

Errors render a small HTML message: ”Undefined portlet” if the ID is unknown, ”Error displaying portlet” if the view raises. The ”Choose Portlet” list contains the enabled portlets; for a non-administrator, only those with permission `run` (see Permissions). A dashboard can be exported and imported as YAML (`*_dashboardpage.yml`), assigned to users or groups, and ranked.

5.2 Main view

`<id>.erb` is rendered inside the layout `individual_portlet`, which is loaded into the frame. Variables available:

Variable	Content
`@portlet`	The portlet object, with its runtime values.
`@session_user`	The ID of the logged-in user.
`@portlet_id`, `@portlet_name`	ID and name.
`@frame_id`, `@frame_html_id`	The frame, and the HTML `id` of its container, for example `dashboard_2` in a dashboard page.
`@parameters`	The hash of runtime values.
`params`, `session`	As in any controller view.

Rules:

- Escape every value with `h` (or `<%= %>`, which escapes in Rails views). Work order names, user names and variables are user data.

- Put the heading in a `<main>` with `aria-labelledby` as the shipped portlets do; use the Bootstrap 5 classes `container-fluid`, `row fluid-margin`, `page_title`, `orch_table_basic`, `table table-condensed`.
- Several portlets are on the same page. Scope every CSS rule with `#<%= @frame_html_id %>`, and every `id` you create with a name that is unique to the portlet. Orchestrator does not rewrite CSS.
- jQuery 3.7.1, jQuery UI, Bootstrap 5, select2, SweetAlert2 and clipboard.js are already loaded by the application layout.
- 14 of the 22 shipped main views contain a `<script>` (`custom_portlet` has no main view). The script runs when the frame content is inserted.
- Check permissions in the view for data that depends on them: `check_permission({ user: @session_user }, workflow, :run)`, `authorized_workflows_selector(session[:user], ...)`, `User.admin?(session[:user])`.

5.3 Configuration partial

`_<id>.erb` is rendered by the Dashboard configuration (`frames/show`) from the `configuration_link` of the frame, as partial `<id>/<id>`. It contains `<li>` elements. The form buttons are Save Configuration, Reset Frame and Preview.

Use the helpers of `PortletsHelper`:

Helper	Output
`portlet_input_name(portlet, name)`	`name="__<name>"`
`portlet_input_value(portlet, name)`	`value="..."`; when the value equals the default, it outputs `placeholder="default: X" value=""`.
`portlet_input_name_and_value(portlet, name)`	Both. Most used (29 shipped uses).
`style_short`, `style_long`	Width 120px, 320px.
`draw_portlet_text_input(...)`, `draw_portlet_text_list_element(...)`	Text input, bare or in `<li>`.
`draw_portlet_flag_input(...)`, `draw_portlet_flag_list_element(name, label)`	Check box with a hidden `false` field.
`draw_portlet_selection_input(...)`, `draw_portlet_selection_list_element(...)`	Select list.
`double_password_configurator`	Password with confirmation.
`get_label`	Label of a field.

Other usable helpers: `authorized_workflows_selector(user_id, name, opts, existing)`, `script_for_variables_hash`, `draw_variables_hash`, `check_permission`.

The form field name for parameter `p` is `__p`. `FramesController#configure` keeps the fields that match `^[0-9]*__(.*)` and stores their text in the frame as `link_parameters`; values are converted with their type at each display. A check box without the hidden field cannot be unchecked; use `draw_portlet_flag_*`.

5.4 JavaScript

`ajaxUpdate(url, method, options)` (file `application.js`) requests a URL and inserts the HTML response in an element:

Option	Meaning
`target_id`	jQuery selector of the element that receives the response. Prefix it with `#<%= @frame_html_id %>`.
`beforeSuccess`, `success`, `error`, `complete`	Callbacks. Without `error`, a generic error message is shown.
`morphUpdate`	Update the DOM in place instead of replacing it.

New `<link rel="stylesheet">` elements found in the response are moved to `<head>`. `portlet_to_modal(url, method)` shows the response in the modal dialog `#modal_dialog` (6 shipped uses). `xhr-define.js` adds the `X-CSRF-Token` header to every XHR request that is not a GET, so a POST must be sent by `ajaxUpdate` or by `$.ajax`. A POST without a valid token is not only rejected: Orchestrator invalidates all the sessions of the user and redirects to the logon page. A plain `<form>` POST without the `authenticity_token` field therefore logs the user out.

5.5 Actions

A portlet action is a public method of `<Class>Controller < PortletsController`, in the portlet file.

URL helper:

```
<%= raw portlet_action_path(portlet: "hello_banner", portlet_action: "ping",
      frame_id: @frame_id, frame_html_id: @frame_html_id) %>
```

It generates `GET` or `POST /aspera/orchestrator/portlet_action?portlet=<id>&portlet_action=<method>&frame_id=..&frame_html_id=..`.

`PortletsController#portlet_action`:

1. Loads the portlet, enabling it if it is defined but not loaded.
2. Merges the `link_parameters` of the frame into `params`.
3. Creates `<Class>Controller`, and sets `request`, `response`, `frame_id`, `frame_html_id`, `portlet`, `user` and `params`.
4. Calls `controller.process(params[:portlet_action])`, and copies the body, status, content type and headers to the response.

In the action: `@portlet`, `@frame_id`, `@frame_html_id`, `params`, `session`, `current_user`, `render plain:`, `render partial:`, `render html:`. Views are looked up as `<id>/<partial>`. `helper_method` works (see `queue_management`). Private methods cannot be called as actions.

Orchestrator checks no permission for an action: check it in the action, see Security of actions.

Chapter 6

Installation and updates

6.1 Installing a portlet

Stock folder `/opt/aspera/orchestrator/portlets` is overwritten by upgrades; use the user folder.

Loading order: the user folder `<config_dir>/portlets/<id>` has priority over the stock folder. `Portlet.available` lists both; a user folder counts as soon as it contains a `.rb` file.

1. Put the folder in `/opt/aspera/var/config/orchestrator/portlets/<id>`, or import `<id>.portlet`.
2. Open Engine > Portlets (permission `engine`) and enable the portlet. Enabling runs `Portlet.attempt_load(id)`: it loads `<id>.rb`, checks the class, and creates the row with `active = true`.
3. To enable a portlet at every start, add its ID to the OrchConfig setting `default_portlet_list` (comma-separated); `config/application.rb` loads these portlets after initialization. Setting `auto_activate_portlets` enables portlets when they are assigned to users by dashboard.

A portlet is loaded only in the web process (not in the engine or the workers). Disabling destroys the row and removes the class constant. The controller constant stays defined until restart. A disabled portlet is enabled again, with a new row, as soon as a frame displays it or calls an action on it, because `view` and `portlet_action` autoload.

6.2 Importing and exporting

- Export (Export in the portlet list) creates `<id>.portlet`, a gzip tar of the portlet folder.
- Import (`Portlet.uncompact`) accepts `.portlet` (a tar) or `.yml` (a custom portlet). The tar must contain the folder at the top level. The archive name, without the extension, must equal the portlet ID: `hello_banner_1.0.0.portlet` extracts the folder `hello_banner`, overwriting the files of an installed `hello_banner`, then fails to load `hello_banner_1.0.0`.
- If a portlet with the same ID exists in the stock folder, the import renames the new one to `<id>_version_N`: the folder, the file names, the class name (by text replacement) and the name, which becomes "... (Version: N)" (only for a name in double quotes). Do not use the string of the class name in text that must not change, and do not rely on a hard-coded `portlet: "<id>"` in views: update the renamed copy yourself.
- If the portlet exists in the user folder, the import does the same renaming, but extracts the archive over the installed folder before renaming it: the installed portlet loses its files. It keeps working until the next restart, then its frames show "Undefined portlet". Never import an archive of a portlet installed in the user folder; see Updating a portlet.
- Imports go through `<run_dir>/portlet/import`, exports through `<run_dir>/portlet/export`.

6.3 Updating a portlet

1. Replace the files in the user folder. Do not import a new archive with the same ID: it breaks the installed portlet (see Importing and exporting).
2. Increase `:version` in `metadata.yml`.
3. Restart the web server: `systemctl restart orchestrator_server` (or `orchestrator restart` for all services). In production, `enable_reloading` is `false` and `eager_load` is `true`; Orchestrator does not reload a portlet

in the running process. A disable followed by an enable runs `load` again on `<id>.rb`, so the model and the methods of the controller change, but the views stay cached until restart; it also changes the row ID and loses permissions.
4. Check the frames that use the portlet: they keep their stored parameters. A parameter removed from `parameters_spec` stays in the frame and is ignored.

6.4 Upgrading a legacy portlet

`script/update_portlet.rb <directory>` upgrades a portlet written before 4.1.0:

- creates `metadata.yml` from a ”Revision history” comment;
- rewrites `def name` and `def description` into `set_portlet_name_and_desc`;
- rewrites `< ApplicationController` into `< PortletsController`;
- rewrites `AbInitio::X` into `OrchConfig.x`;
- rewrites `/portlet_action/<p>/<a>` URLs into `portlet_action_path(...)`.

It skips a folder that has a `metadata.yml`, and does not scope CSS rules to the frame: do that by hand.

Chapter 7

Testing and debugging

- Check the syntax: `ruby -c <id>.rb`.
- Display the portlet in a frame with Preview, before saving the configuration.
- Read `/opt/aspera/orchestrator/log/production.log`; write to it with `Rails.logger` in the portlet.
- Call an action in the browser: `/aspera/orchestrator/portlet_action?portlet=<id>&portlet_action=<method>&frame_id=1&frame_html_id=f1` (logged in).
- Call `GET /aspera/orchestrator/api/portlet_version[/<name>]?format=json` (or `xml`), with API credentials, to read the version of a portlet from `metadata.yml`, or of all portlets without `<name>`. `<name>` is the portlet name, URL-encoded (`Hello%20banner`), not the ID. The result is wrong in 4.1.6 (see Known issues).

Symptom	Cause
Enabling fails with ”definition of the portlet is missing”	Class name different from `id.camelize`, or file not `<id>.rb`.
Import fails, and the files of the installed portlet changed	Archive name differs from `<id>.portlet`.
Choosing the portlet in a frame fails (HTTP 500, ”Missing partial” in the log)	`_<id>.erb` is missing.
After a restart, frames show ”Undefined portlet”	An archive of the same ID was imported: the folder was moved to `<id>_version_N`.
Name in the list shows Ruby code	`self.name` is computed; use a literal string.
A parameter is always the default	The field is blank, or the name is not `__<name>`.
A flag is always `false`	Value is not the text `true`. Use `draw_portlet_flag_*`.
`NoMethodError` on a parameter	Not in `parameters_spec`, no default, and not in `link_parameters`.
A parameter named `name` is wrong	Shadowed by the column. Use `runtime_value('name')`.
Code change has no effect	The web server was not restarted.
Permissions disappear after disable	New row ID after enabling (see Permissions).
Styles of another portlet change	CSS not scoped by `#<%= @frame_html_id %>`.
A POST action logs the user out	CSRF token missing: use `ajaxUpdate` or `$.ajax`.
A flag is `nil` instead of `false`	Its default is `false`; test it with `if`.
User cannot select a portlet in Choose Portlet	Known issue with non-administrators.

7.1 Known issues

Each was reproduced on a 4.1.6 server, except the regular expression on `link_parameters`, reproduced with the unmodified classes of the 4.1.6 package.

Issue	Effect	Work-around
`Portlet.version` is a class method that reads `first_or_initialize.portlet_id`.	`api/portlet_version` returns the version of the first portlet row for every portlet. On the test server, all portlets, `Hello banner` 1.0.0 and `Queue Manager` 0.0.9 included, were reported as 0.0.3, the version of `aspera_service`.	Read `metadata.yml` directly.
`Entitlement.matching_permissions` returns an array of parts and the list tests complete principals.	A non-administrator with an explicit `run` permission does not see the portlet in Choose Portlet.	Assign the portlet to the user with the administrator, through a dashboard.
Portlet controllers inherit the public methods of `PortletsController`.	`portlet_action=disable` and similar are reachable without a permission check.	Check permissions in each action; do not expose sensitive operations.
The row ID changes when a portlet is disabled and enabled.	Permissions are orphaned.	Avoid disable and enable; grant permissions again.
`.portlet` file name must equal the ID.	Import fails for names such as `<id>_1.0.0.portlet`, after overwriting the files of an installed `<id>`.	Rename the file.
Import of an ID present in the user folder extracts over it, then moves it to `<id>_version_N`.	The installed portlet loses its files, and breaks at the next restart.	Update by replacing the files; do not import.
`link_parameters` is parsed with a regular expression in `portlet_action`.	`"` and `\` in a value are doubled.	Avoid these characters.
A POST without CSRF token calls `redirect_to_login`.	All sessions of the user are invalidated.	Send POST requests with `ajaxUpdate` or `$.ajax`.

Chapter 8

Security

- A portlet runs with the rights of the Orchestrator web process and can read the whole database. Review the code of every imported `.portlet` and `.yml` file before enabling it.
- Import requires the permission `engine`; grant it only to trusted administrators. In 4.1.6, enable, disable and export check no permission: any logged-in user can call them (see Security of actions).
- Escape all output (`h`). Validate every parameter and every `params` value used in queries, commands and file paths.
- Check permissions inside each action and view; see Security of actions.
- Do not store passwords in parameters: they are saved in the frame as clear text. Use `double_password_configurator` only for values that the portlet must hold, and prefer a stored credential.
- Custom portlets run `eval`: restrict the permission `engine`.

8.1 Permissions

Each portlet has entitlements with principals `Aspera:Orchestrator:Portlet:<row id>:run`, `:view` and `:edit`, edited with Edit permissions in Engine > Portlets. Administrators see all portlets.

Limits of 4.1.6:

- The principal contains the database row ID, not the portlet ID. Disabling a portlet destroys the row; enabling it creates another row with another ID, and the permissions stay attached to the old one.
- `Entitlement.matching_permissions` returns the parts of the principal as an array, while the portlet list expects complete principals. As a result, a non-administrator with an explicit `run` permission does not see the portlet in Choose Portlet (see Known issues).

Do not rely on portlet permissions for confidentiality; check permissions on the data, as in Security of actions.

8.2 Security of actions

`<Class>Controller` inherits the public methods of `PortletsController`, and `portlet_action` calls any public method by name. In 4.1.6, `portlet_action=disable`, `enable`, `export`, `upload`, `import`, `revoke`, `update_permissions`, `view` and `index` are reachable through a portlet controller. Only some of them check a permission, and `disable`, `enable`, `export` and `view` check none. The same methods of `PortletsController` are also reachable directly (`POST portlets/disable`, `POST portlets/enable`, `GET portlets/export`) by any logged-in user: only the list page, Import Portlet and the upload check the permission `engine`.

Therefore:

- Orchestrator checks only that the user is logged in, the CSRF token and the host name. It never checks a permission for a portlet action.
- Check the permission in every action that reads sensitive data or changes something: `check_permission({ user: session[:user] }, workflow, :run)` or `User.admin?(session[:user])`, and respond with an error otherwise.
- Do not name an action like an existing `PortletsController` method.

- Parameters of the frame, as they reach an action, are parsed from the stored text with a regular expression. A value that contains `"` or `\` comes back with extra backslashes. Do not put such values in parameters used by an action, or read them with `@portlet.runtime_value`.

Chapter 9

Checklist

1. Choose the portlet ID (snake_case, no acronym), the class `id.camelize`, and the folder name.
2. Create `<id>.rb`, `<id>.erb`, `_<id>.erb` and `metadata.yml`.
3. Set name and description in `set_portlet_name_and_desc` with literal strings, called by `after_initialize` and `before_save`.
4. Declare `refresh`, and every parameter in `parameters_spec`, with a default in `defaulted_parameters`.
5. Escape output, scope CSS with `#<%= @frame_html_id %>`, and give every `id` a unique name.
6. For an action, add `<Class>Controller < PortletsController`, check the permission in the action, and call it with `portlet_action_path` and `ajaxUpdate`.
7. Pack as `<id>.portlet` (name equal to the ID), import, enable, and configure a frame. After each code change, replace the files in the user folder (do not import again) and restart the web server.

Chapter 10

Annex

10.1 Comparison with plugins

Portlets and plugins are two applications of the same design.

Common points:

- A component is a folder with `<name>.rb`, `metadata.yml` with `:revision_history` and `:minimum_server_version`, ERB views, and optional CSS and JavaScript.
- A stock folder in `/opt/aspera/orchestrator` and a user folder in `<config_dir>` (`actions` or `portlets`); the user copy has priority.
- The class is an Active Record model whose table is the registry of enabled components; the Engine area has a page with a slider to enable or disable each component, and the permission `engine`.
- Export and import as a gzip tar (`.plugin`, `.portlet`) with `File.tgz` and `untgz`, and renaming on collision.
- Version reading with `SafeYaml.load_file('metadata.yml')[:revision_history]`, sorted as `[major, minor, revision, patch]`.
- The same libraries: `MultiType` (`TYPE_*`), `Payload.expand`, `OrchConfig`, `Rails.logger`, `BasicUpload` (also used by portlet `csv_editor`) and `ModuleSSHTools` (also included by portlets `aspera_service` and `transfer_console`).
- A component can have its own controller class in its folder.

Differences:

Topic	Plugin	Portlet
Table	One table per plugin, created by a migration.	One shared table `portlets`, no migration.
Settings	Attributes, in the template row.	Parameters, in the frame (`link_parameters`).
Naming	`tableize` and `classify` must round-trip.	`camelize` only.
Loading	Zeitwerk, in engine, workers and web.	`load` on demand, web process only.
Compatibility check	Version, beta and dependencies checked.	None.
Controller	Own `<name>_controller.rb`, in the `skip_authenticity_token` list.	`<Class>Controller < PortletsController`, called by `portlet_action`; CSRF always applies.
Permission	Entitlement of the template (`Action`).	Entitlement of the portlet row (`Portlet:<row id>`).
Runtime object	Kept between `execute` and `check_status`.	Rebuilt at every display.

10.2 Custom portlets

A custom portlet is defined in the web interface, with no file: Engine > Portlets > Define Custom Portlet. Portlet `custom_portlet` (stock) must be enabled first.

A definition (`custom_portlet_definitions`) has a name, a display name, a version, a description, parameters (typed), a model (Ruby) and a view (ERB, rendered inline). The model is evaluated with `eval` as methods of the portlet.

- It runs Ruby and ERB with the rights of Orchestrator. The forms Define Custom Portlet and edit require the permission `engine`, but in 4.1.6 the create, update and delete requests themselves check no permission.
- Definitions are exported as `CustomPortlet_<id>_<version>.yml` and imported in the same page. Importing an existing name increases the version and deactivates the previous one.
- IDs are `c_<n>`.

Use it for a quick prototype. Write a file portlet for anything that is kept, versioned, tested and reviewed.

10.3 Patterns in the shipped portlets

Counts in the 23 portlets of Orchestrator 4.1.6 (versions 0.0.2 to 0.1.2, all with `:minimum_server_version: 4.1.0`):

Pattern	Count
Occurrences of `TYPE_STRING`	44
Occurrences of `TYPE_INT`	40
Occurrences of `TYPE_FLAG`	36
Occurrences of `TYPE_DATE`	3
Occurrences of `TYPE_HASH`	3
`portlet_input_name_and_value`	29
`style_short`	28
ERB files with `<script>` (in 14 portlets)	18
`portlet_input_name`	12
Portlets with a controller	10
`portlet_to_modal`	6
`draw_portlet_*`	6
`ModuleSSHTools`	2

Shipped portlets: `aspera_service`, `assigned_tasks`, `csv_editor`, `custom_portlet`, `file_browser`, `filter_work_orders`, `inline_journals`, `load_utilization`, `lookup_editor`, `monitors_dashboard`, `orchestrator_information`, `processes_view`, `queue_management`, `resource_management`, `step_activity`, `step_performance_stats`, `step_stats`, `table_details_viewer`, `transfer_console`, `user_tasks`, `workflow_performance_stats`, `workflow_stats`, `workorder_activities`.

10.4 Learning Ruby and Rails

- Ruby documentation
- Rails guides: Action View, Action Controller.

End of document